

Spotify Data Engineering on AWS

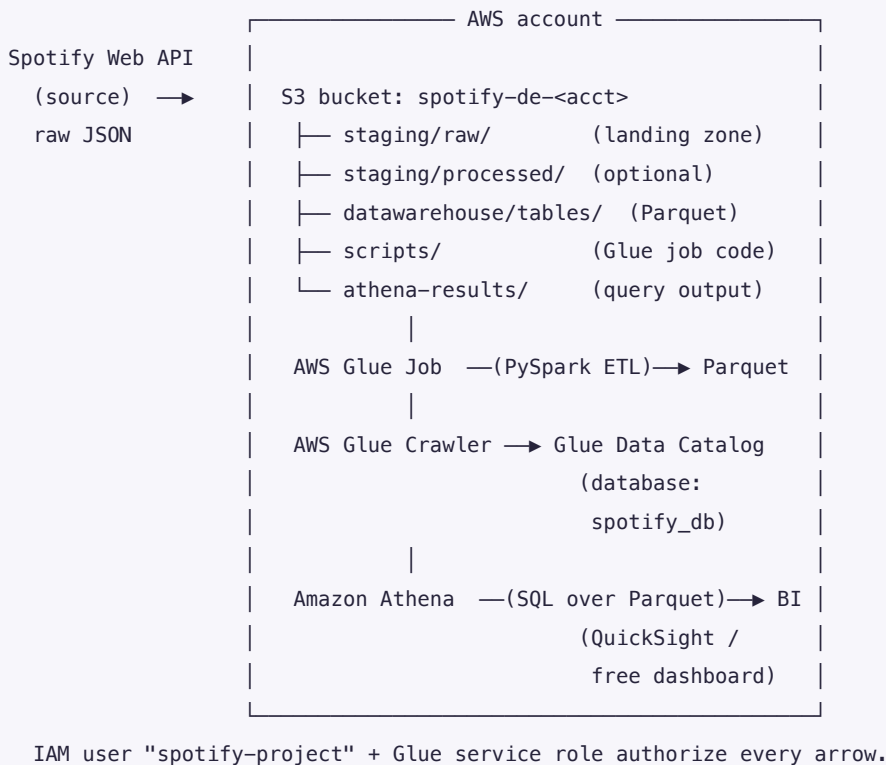
Complete Build & Interview-Prep Guide

IAM · S3 · Glue · Athena · QuickSight | reproduce-it-yourself, errors & fixes, interview Q&A

A from-scratch, reproduce-it-yourself guide to the pipeline we built. Written so you can rebuild it **without any assistant** on your next project, understand *why* each service is there, answer **interview questions** about it, recognize the **errors** that come up (and their fixes), and know what a **data engineer does with this in real life**.

Account/region used here: account `810995308424`, region `us-west-2`. Swap in your own values everywhere you see them.

0. The architecture at a glance



Pattern name: a serverless **data lake** + **"lakehouse" query layer**. Storage (S3) is separate from compute (Glue, Athena). Nothing runs when idle → you pay per job/query, not per hour.

1. Prerequisites

- An AWS account (root access to bootstrap the first IAM user).
- **AWS CLI v2** installed: `aws --version`.
- Basic shell. Everything below is copy-pasteable.

Golden rule you'll be asked about in interviews: *never* use the **root** user for daily work. Root is only for account-level tasks (billing, closing the account, creating the first admin). We create an IAM user immediately and use that.

2. IAM — identity & permissions (who is allowed to do what)

What IAM is

Identity and Access Management controls *authentication* (who you are) and *authorization* (what you can do). Core objects:

- **User** — a person or app identity (long-lived).
- **Access key** — the user's programmatic credential (ID + secret) for CLI/SDK.
- **Policy** — a JSON document listing allowed/denied actions on resources.
- **Role** — an identity a *service* assumes temporarily (no long-lived keys). Glue

uses a role; users use keys.

2.1 Create the project user

```
aws iam create-user --user-name spotify-project \
  --tags Key=project,Value=spotify-data-engineering
```

2.2 Attach permissions (managed policies)

```
# Broad admin (fine for a solo learning account; see "least privilege" note below)
aws iam attach-user-policy --user-name spotify-project \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess

# Service-specific ones we used
aws iam attach-user-policy --user-name spotify-project \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
aws iam attach-user-policy --user-name spotify-project \
  --policy-arn arn:aws:iam::aws:policy/AWSGlueConsoleFullAccess
aws iam attach-user-policy --user-name spotify-project \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSQuicksightAthenaAccess
aws iam attach-user-policy --user-name spotify-project \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSQuickSightDescribeRDS
```

Find any managed policy's exact ARN:

```
aws iam list-policies --scope AWS --query "Policies[?contains(PolicyName,'Glue')].
[PolicyName,Arn]" --output text
```

2.3 Create access keys and store them safely

```
aws iam create-access-key --user-name spotify-project
```

Take the returned `AccessKeyId` + `SecretAccessKey` and save them into a **named CLI profile** (never paste secrets into code or chat):

```
aws configure set aws_access_key_id AKIA... --profile spotify-project
aws configure set aws_secret_access_key ..... --profile spotify-project
aws configure set region us-west-2 --profile spotify-project
aws configure set output json --profile spotify-project

# use it:
export AWS_PROFILE=spotify-project
aws sts get-caller-identity # verify: returns the user's ARN
```

2.4 Least privilege (the "right" way, for real projects)

AdministratorAccess is convenient but over-privileged. In production you'd attach only what's needed, e.g. an inline policy scoped to **one bucket**:

```
{ "Version":"2012-10-17","Statement":[
  {"Effect":"Allow","Action":["s3:GetObject","s3:PutObject","s3:ListBucket"],
    "Resource":["arn:aws:s3:::spotify-de-<acct>","arn:aws:s3:::spotify-de-<acct>/*"]}]}
```

3. Amazon S3 — the storage layer (the data lake)

What S3 is

Object storage: infinitely scalable, 11 nines of durability, pay-per-GB. **Buckets** are globally-unique top-level containers; **objects** are files; **prefixes** (the `folder/` in a key) *simulate* folders — S3 is actually flat.

3.1 One bucket, many prefixes (the layout that matters)

⚠️ **You cannot nest buckets inside a folder.** Buckets are always top-level. The "staging vs warehouse" split is done with **prefixes inside one bucket**:

```
export AWS_PROFILE=spotify-project
REGION=us-west-2; ACCT=<acct>; BUCKET=spotify-de-$ACCT

# create (note: outside us-east-1 you MUST pass LocationConstraint)
aws s3api create-bucket --bucket "$BUCKET" --region "$REGION" \
  --create-bucket-configuration LocationConstraint="$REGION"

# security: block ALL public access
aws s3api put-public-access-block --bucket "$BUCKET" \
  --public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

# versioning: protects against overwrite/delete mistakes
aws s3api put-bucket-versioning --bucket "$BUCKET" --versioning-configuration Status=Enabled

# create the prefix structure (0-byte placeholders)
for k in staging/raw/.keep staging/processed/.keep datawarehouse/tables/.keep; do
  printf ' ' | aws s3 cp - "s3://$BUCKET/$k"; done
```

3.2 Upload data

```
aws s3 cp myfile.json "s3://$BUCKET/staging/raw/"
aws s3 ls "s3://$BUCKET/staging/raw/" --recursive --human-readable
```

Why this layout is good (interview-ready reasoning)

- **Zone separation** (`raw` / `processed` / `warehouse`) = the classic

medallion / bronze-silver-gold pattern. Raw is immutable (you can always reprocess); curated is derived.

- **One bucket** keeps IAM, lifecycle rules, and cost tracking simple; prefixes give

logical separation without extra bucket sprawl.

- **Parquet in the warehouse zone** (columnar, compressed) → Athena scans far less

data = faster + cheaper than JSON/CSV.

4. AWS Glue — the ETL engine + the catalog

Glue is three things people conflate:

1. **Glue ETL Job** — serverless **Apache Spark** that transforms data.
2. **Glue Data Catalog** — a metadata store (databases → tables → schemas). It's the

Hive metastore that Athena, Redshift Spectrum, and EMR all read.

1. **Glue Crawler** — a process that scans S3, infers schema, and populates the Catalog.

4.1 Glue needs a *service role* (not the user's keys)

```
ROLE=AWSGlueServiceRole-spotify
cat > glue-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"glue.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
aws iam create-role --role-name "$ROLE" \
  --assume-role-policy-document file://glue-trust.json
aws iam attach-role-policy --role-name "$ROLE" \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
aws iam attach-role-policy --role-name "$ROLE" \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
```

Why a role, not keys? Services can't safely hold long-lived secrets. A role lets

Glue get **temporary** credentials via `sts:AssumeRole`. The **trust policy** says *who* may assume it (the Glue service); the **attached policies** say *what* it can do.

Name it `AWSGlueServiceRole-` * — the Glue console's `iam:PassRole` permission is scoped to that prefix.

4.2 The ETL script (PySpark)

Full script: `src/glue/spotify_staging_to_warehouse.py`. It reads nested JSON, **explodes** arrays into rows, selects/casts clean columns, adds a `processed_at` audit column, and writes **Parquet** (overwrite mode). Upload it to S3:

```
aws s3 cp src/glue/spotify_staging_to_warehouse.py "s3://$BUCKET/scripts/"
```

4.3 Create and run the job

```
ROLE_ARN=$(aws iam get-role --role-name AWSGlueServiceRole-spotify --query 'Role.Arn' --output text)
JOB=spotify-staging-to-warehouse
aws glue create-job --name "$JOB" --role "$ROLE_ARN" \
  --glue-version "4.0" --number-of-workers 2 --worker-type G.1X \
  --command
"Name=glueetl,ScriptLocation=s3://$BUCKET/scripts/spotify_staging_to_warehouse.py,PythonVersion=3" \
  --default-arguments "{\"--JOB_NAME\":\"$JOB\", \"--S3_BUCKET\":\"$BUCKET\", \"--
STAGING_PREFIX\":\"staging/\", \"--WAREHOUSE_PREFIX\":\"datawarehouse/\", \"--
TempDir\":\"s3://$BUCKET/glue-temp/\"}"

RUN=$(aws glue start-job-run --job-name "$JOB" --query JobRunId --output text)
aws glue get-job-run --job-name "$JOB" --run-id "$RUN" --query 'JobRun.JobRunState' # poll until
SUCCEEDED
```

- **Worker type** **G.1X** = 4 vCPU / 16 GB per worker; **2** workers is the practical

minimum. **Glue 4.0** = Spark 3.3.

- Cost model: billed per **DPU-hour** by the second (1 min minimum). Two workers for

~2 min ≈ pennies.

4.4 Crawler → Data Catalog

```
DB=spotify_db; CRAWLER=spotify-warehouse-crawler
aws glue create-database --database-input "{\"Name\":\"$DB\"}"
aws glue create-crawler --name "$CRAWLER" --role AWSGlueServiceRole-spotify \
  --database-name "$DB" \
  --targets "{\"S3Targets\":[{\"Path\":\"s3://$BUCKET/datawarehouse/tables/\"}]}"
aws glue start-crawler --name "$CRAWLER"
aws glue get-crawler --name "$CRAWLER" --query 'Crawler.State' # poll until READY
aws glue get-tables --database-name "$DB" --query 'TableList[].Name'
```

The crawler points at **datawarehouse/tables/** and creates one table per sub-folder (**tracks**, **artists**, **albums**), inferring column names and types from the Parquet.

5. Amazon Athena — serverless SQL

What Athena is

Presto/Trino engine that runs ANSI SQL directly on S3 files, using the Glue Catalog for schemas. No servers, no loading — **query-in-place**. Billed **~\$5 per TB scanned**.

5.1 Set an output location and query

```
OUT="s3://$BUCKET/athena-results/"
aws athena start-query-execution \
  --query-string "SELECT t.track_name, ar.artist_name, t.popularity
                  FROM tracks t JOIN artists ar ON ar.artist_id=t.primary_artist_id
                  ORDER BY t.popularity DESC" \
  --query-execution-context "Database=$DB" \
  --result-configuration "OutputLocation=$OUT"
# then get-query-execution (poll state) → get-query-results
```

Ready-made queries: [sql/athena_queries.sql](#).

Cost levers you should name in an interview

- **Columnar (Parquet) + compression** → scans only needed columns.
- **Partitioning** (e.g. [s3://.../tracks/dt=2026-08-05/](#)) → Athena skips irrelevant

partitions. Register with partition projection or [MSCK REPAIR TABLE](#).

- **CTAS / compaction** → merge many small files into fewer big ones.

6. Amazon QuickSight — BI (and why we used a free alternative)

QuickSight is AWS's managed BI/dashboard tool (Athena/RDS/Redshift/S3 sources). It is **paid** (Standard ~\$9, Enterprise ~\$18 per author/mo; 30-day free trial).

We chose a free, self-contained HTML dashboard instead (see [dashboards/spotify_dashboard.html](#)) because the project is for learning and QuickSight's first-time signup was blocked (see §7). The dashboard visualizes the same Athena output: track popularity, artist followers, genre frequency.

To use QuickSight for real: sign up once **in the console** (the CLI can't do the first signup — see §7), then in *Manage QuickSight* → *Security & permissions* grant it **Athena** + your **S3 bucket**, then build datasets on the [spotify_db](#) tables.

7. Exceptions we hit & how we resolved them (real debugging)

#	Symptom	Root cause	Fix
1	<code>InvalidClientTokenId</code> right after creating an access key	IAM eventual consistency — new keys take a few seconds to propagate globally	Retry <code>sts get-caller-identity</code> with a short backoff (5–40s). Not an error in your code.
2	<code>create-bucket</code> fails / bucket name rejected	Bucket names are globally unique across all AWS accounts	Suffix with account id or a random token: <code>spotify-de- <acct></code> .
3	"Can I put the 2 buckets in one folder?"	Buckets can't be nested ; S3 has no folder-of-buckets	Use one bucket with <code>staging/</code> and <code>datawarehouse/</code> prefixes.
4	Glue job needs to read/write S3 but user keys don't apply	Glue runs as a service , uses a role , not your user keys	Create <code>AWSGlueServiceRole-*</code> with a trust policy for <code>glue.amazonaws.com</code> .
5	Spark reads 0 rows / wrong schema from the JSON	Files are multi-line JSON objects, not newline-delimited	<code>spark.read.option("multiLine","true").json(...)</code> then <code>explode()</code> the arrays.
6	<code>create-account-subscription</code> → <code>PreconditionNotMetException: account does not have subscription to AmazonQuickSight</code>	The first QuickSight enablement must go through the console (it does a Marketplace step); the API only works <i>after</i> that	Sign up once in the console, or (our choice) use a free dashboard.
7	<code>psql</code> earlier looped on password prompts	A second Postgres server already held port 5432 with password auth	Run the new server on another port / point the client at the right one. <i>(from the earlier Postgres setup)</i>
8	Region mismatch: BI/Athena "sees no data"	QuickSight/Athena must be in the same region as the Glue Catalog & S3	Keep everything in one region (<code>us-west-2</code> here).

General debugging muscle: read the *exact* exception name (it tells you the service + category), check **CloudWatch Logs** for Glue jobs (`--enable-continuous-cloudwatch-log`), and remember that many IAM/S3 failures are **propagation delays**, not logic bugs.

8. Why this design is good — and alternatives

Why it's good

- **Decoupled storage & compute** — scale/pay for each independently; reprocess raw

anytime.

- **Serverless** — no clusters to babysit; near-zero idle cost. Great for spiky/onboarding

workloads and for learning.

- **Open formats + open catalog** — Parquet + Glue Catalog aren't locked to one engine;

Athena, Redshift Spectrum, EMR, and Spark all read them.

- **Schema-on-read** — the crawler adapts to evolving data without rigid pre-modeling.

When you'd choose something else

Need	Better fit
Sub-second dashboards, many concurrent BI users	Redshift (or Athena + a BI cache)
Complex, testable transformations, lineage	dbt on Redshift/Snowflake, or Glue + Great Expectations
Streaming / near-real-time	Kinesis / MSK (Kafka) → Flink / Spark Streaming
Heavy, long-running Spark, custom libs	EMR (more control than Glue)
Orchestration of many dependent steps	Airflow (MWAA) or Step Functions , not manual <code>start-job-run</code>
Table-level ACID, upserts, time-travel	Apache Iceberg / Hudi / Delta on S3

Trade-offs to acknowledge (interviewers love this)

- Athena is **pay-per-scan** → costs surprise you without partitioning/compaction.
- Glue crawlers can **mis-infer types** or create tiny-file messes → many teams define

tables explicitly and skip crawlers.

- Serverless cold starts (Glue job startup ~1 min) make it poor for **low-latency** needs.

9. Interview perspective — likely questions & crisp answers

Q: Walk me through your pipeline. A: Spotify JSON lands in S3 `staging/raw` (immutable bronze). A serverless Glue Spark job flattens/cleans it and writes partitioned Parquet to `datawarehouse` (gold). A Glue crawler catalogs it into the Glue Data Catalog; Athena queries it with SQL; BI (QuickSight or a dashboard) sits on top. Storage and compute are fully decoupled.

Q: Why Parquet over CSV/JSON? A: Columnar + compressed → Athena scans only needed columns → faster and cheaper (Athena bills per TB scanned). Also self-describing schema and splittable for Spark.

Q: User keys vs IAM roles? A: Users have long-lived keys for humans/CLI. Services assume **roles** for **temporary** credentials via STS — no secrets to leak/rotate. Glue, Lambda, EC2 all use roles.

Q: How do you control Athena cost? A: Partition the data, store as Parquet, compact small files, select only needed columns, and set per-query/-workgroup data-scan limits.

Q: What's the Glue Data Catalog? A: A managed Hive metastore (databases→tables→schema + partition metadata) shared across Athena, Redshift Spectrum, EMR, and Glue jobs. The crawler populates it.

Q: Idempotency / re-runs? A: The job writes `overwrite`, raw stays immutable, so re-running reproduces the same output. For incremental loads you'd use **Glue job bookmarks** or partition-by-date.

Q: How would you productionize this? A: Orchestrate with Step Functions/Airflow, trigger on S3 events (EventBridge), add data quality checks (Great Expectations / Glue Data Quality), partition + compact, add CI/CD (IaC via Terraform/CloudFormation), monitoring/alerting (CloudWatch), and least- privilege IAM.

10. What a data engineer actually does with this in real life

The demo has 3 rows; production has millions. The real day-to-day around this exact architecture:

1. **Ingestion at scale & incrementally** — pull the Spotify API on a schedule (paged, rate-limited, token-refreshed), land **date-partitioned** raw files (`staging/raw/dt=YYYY-MM-DD/`). Handle retries, backfills, and late data.
 1. **Reliable transforms** — parameterized Glue jobs with **bookmarks** for incremental loads, schema-evolution handling, and unit-tested transformation logic.
 1. **Data modeling** — turn raw into a **star schema** (fact_plays, dim_track, dim_artist, dim_album) or use **dbt** for versioned, tested models with lineage.
 1. **Data quality** — null/range/uniqueness/freshness checks that **fail the pipeline** before bad data reaches BI (Glue Data Quality, Great Expectations, dbt tests).
 1. **Orchestration** — Airflow/Step Functions DAGs so crawl → transform → quality → publish run in order with retries and alerting; triggered by EventBridge on new files.
 1. **Cost & performance** — partitioning strategy, small-file compaction, Parquet/Iceberg, Athena workgroup scan limits, S3 lifecycle rules (raw → Glacier after N days).
 1. **Governance & security** — least-privilege IAM, encryption (SSE-KMS), Lake Formation for table/column permissions, PII handling, audit via CloudTrail.
 1. **Serving** — expose gold tables to analysts (Athena/QuickSight), to apps (an API over Athena/Redshift), or to ML feature stores.
 1. **Observability & IaC** — everything defined as code (Terraform/CloudFormation), dashboards on job success/latency/cost, on-call runbooks.

The demo is the "happy path skeleton." The job is making it ****incremental, tested, orchestrated, observable, secure, and cheap**** at scale — that's what separates a script from a data platform.

11. Full teardown (avoid charges when you're done)

```
export AWS_PROFILE=spotify-project; BUCKET=spotify-de-<acct>
aws glue delete-crawler --name spotify-warehouse-crawler
aws glue delete-job      --job-name spotify-staging-to-warehouse
aws glue delete-database --name spotify_db
aws s3 rb "s3://$BUCKET" --force                # empties + deletes the bucket
aws iam detach-role-policy --role-name AWSGlueServiceRole-spotify --policy-arn
arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
aws iam detach-role-policy --role-name AWSGlueServiceRole-spotify --policy-arn
arn:aws:iam::aws:policy/AmazonS3FullAccess
aws iam delete-role --role-name AWSGlueServiceRole-spotify
# (optional) remove the user's keys/policies/user, and cancel QuickSight if you ever subscribed
```

S3, Glue Catalog, and any QuickSight subscription are the things that keep costing money — delete those first.